



Flexible Non-Linear Regression

A practical guide to numerical predictive modeling using the `daltoolbox` package in R. This section explores powerful methods capable of capturing complex non-linear patterns in data — ideal when linear approaches prove insufficient to model the relationship between features and the target variable.

NON-LINEAR REGRESSION

SVM / SVR

MLP / NEURAL NETWORKS

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br

<https://eic.cefet-rj.br/~eogasawara>

Support Vector Regression (SVR)

SVR is a **kernel-based method** derived from Support Vector Machines (SVM). Unlike traditional regression that minimizes squared errors, SVR fits a function within an **epsilon-insensitive tube** — predictions inside the tube incur no penalty, while violations outside are penalized by the cost parameter. This makes SVR robust to outliers and effective in high-dimensional spaces.

Epsilon-tube (ϵ)

Defines a margin of tolerance around the predicted function. Residuals within $\pm\epsilon$ are ignored — only points outside the tube contribute to the loss function.

Cost (C)

Controls the trade-off between model flatness and tolerance for margin violations. **Higher C** = less tolerance = more complex model.

Kernel Function

Maps inputs to a higher-dimensional feature space, enabling non-linear regression. Common choices: **linear**, **polynomial**, and **RBF** (Radial Basis Function).

Mathematical Formulation

SVR minimizes the following objective, balancing model complexity against margin violations:

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad \text{subject to} \quad |y_i - f(x_i)| \leq \epsilon + \xi_i$$

where $\xi_i, \xi_i^* \geq 0$ are **slack variables** that measure the magnitude of violations outside the epsilon tube, and C controls how heavily those violations are penalized.

References

- Drucker, H. et al. (1997). *Support Vector Regression Machines*. NIPS.
- Chang, C.-C. & Lin, C.-J. (2011). *LIBSVM: A library for support vector machines*. ACM TIST.
- Vapnik, V. (1995). *The Nature of Statistical Learning Theory*. Springer.

reg_svm: Support Vector Regression (SVR)

Setup & Data

```
# Regression SVM

library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test
```

Training & Evaluation

```
# epsilon: error-insensitive margin
# cost: penalty for violations
model <- reg_svm("medv",
  epsilon=0.2,
  cost=40.000)
model <- fit(model, boston_train)

# Training evaluation
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=2.86, smape=0.07, R2=0.97

# Test evaluation
test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=14.66, smape=0.14, R2=0.76
```

Step-by-step

- Define the model with `reg_svm("medv", epsilon=0.2, cost=40)`, where `medv` is the target variable
- `epsilon`: width of the error-insensitive margin around predictions
- `cost`: penalty applied for margin violations
- Training $R^2=0.97$ and test $R^2=0.76$ — performance comparable to Random Forest
- The default kernel is applied when no kernel is explicitly specified

❏ **Key concept:** SVR fits a function within an error-insensitive margin of `epsilon`, penalizing violations through `cost`. The main hyperparameters are **epsilon**, **cost**, and **kernel** — each controlling a different aspect of the model's flexibility and tolerance. Ref: Drucker et al. (1997).

Multilayer Perceptron (MLP) – Neural Network for Regression

MLP is a feedforward neural network composed of an **input layer**, one or more **hidden layers**, and an **output layer**. Each neuron applies a non-linear activation function (e.g., sigmoid or ReLU), allowing the network to approximate arbitrarily complex functions. In regression, the output layer produces a single continuous value. MLP is especially powerful when the relationship between predictors and the target is highly non-linear and cannot be captured by simpler models.

Architecture

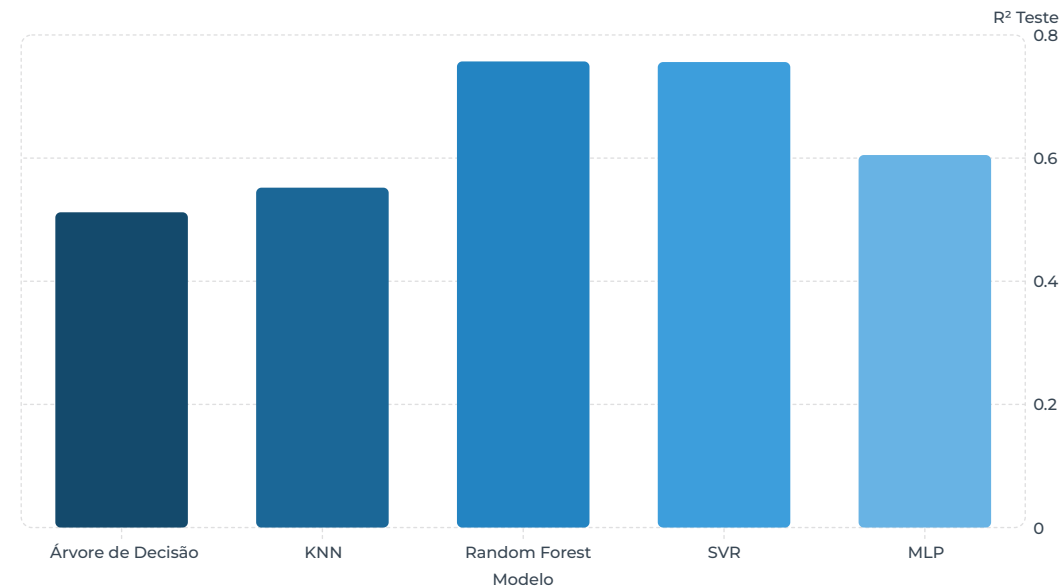
Input layer → hidden layer(s) → output layer. Each layer applies a weighted sum followed by a non-linear activation function. The size parameter controls the number of units in the hidden layer, determining the model's representational capacity.

Regularization (decay)

L2 regularization (weight decay) penalizes large weights, reducing overfitting. The decay parameter controls the strength of this penalty — higher values enforce stronger regularization and simpler weight configurations, improving generalization to unseen data.

Training

Weights are optimized via **backpropagation** using gradient descent. MLP is sensitive to hyperparameter choices (size, decay, learning rate), making careful tuning essential for achieving good generalization performance.



Comparison of test R^2 across all models. Random Forest and SVR lead in generalization; MLP's lower test R^2 reflects sensitivity to hyperparameter tuning.

References

Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Oxford University Press.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors*. Nature, 323, 533–536.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

reg_mlp: Multilayer Perceptron for Regression

Setup and Data

```
# Regression MLP

library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test
```

Training and Evaluation

```
# size: hidden units
# decay: L2 regularization
model <- reg_mlp("medv",
  size=5,
  decay=0.54)
model <- fit(model, boston_train)

# Training evaluation
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=6.53, smape=0.10, R2=0.93

# Test evaluation
test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=23.75, smape=0.17, R2=0.61
```

Step-by-step

- Define the model with `reg_mlp("medv", size=5, decay=0.54)`
- `size`: number of units in the hidden layer
- `decay`: L2 regularization to prevent overfitting
- Training $R^2=0.93$ vs. test $R^2=0.61$ — a larger gap than SVM and Random Forest
- MLP is sensitive to hyperparameter choice; tuning is strongly recommended

❏ **Key concept:** MLP is a neural network with hidden layers. `size` controls model capacity and `decay` controls regularization. The **Experiment Line remains identical** to previous examples — only the model changes. Ref: Bishop (1995).

Summary: daltoolbox Demo Tutorial

Unified Experimental Workflow

A single cycle — split → fit → predict → evaluate — applies consistently across classification, regression, and clustering tasks. The daltoolbox enforces this discipline through a consistent API: `fit()`, `predict()`, and `evaluate()` work the same way for every learner.

Progressive Skill Building

The 13 tutorials form a cumulative sequence: from the first experiment and sampling strategy, through data quality, normalization, baselines, metrics, model comparison, hyperparameter tuning, and pipeline integrity, to regression, clustering, visual reporting, and custom S3 extension. Each step builds on the previous — skipping ahead risks misinterpreting results.

Reproducibility and Extensibility

Fixed random seeds, preprocessing fitted on training data only, and structured metric tables ensure reproducible experiments. The S3 extension pattern allows any external R package to be integrated without modifying the core workflow. New learners only need `fit()` and `predict()` — the rest of the framework remains unchanged.

References & Additional Sources

Academic references and online resources to explore the DAL Toolbox and Experiment Lines further.

Online Resources



CRAN Package – daltoolbox

cran.r-project.org/web/packages/daltoolbox

DAL Toolbox – GitHub

github.com/cefet-rj-dal/daltoolbox

GitHub Repository

github.com/cefet-rj-dal/daltoolbox

Source code, issues, and contributions.

Key Academic References

- Ogasawara, E., Sa, A., Castro, A., Santos, C., Salles, D., Lima, J., Tavares, L., Salles, R. & Saidy, V. (2026). daltoolbox: Leveraging Experiment Lines to Data Analytics. R package version 1.3.767. *CRAN*.
- Ogasawara, E., Paulino, C., Murta, L., Werner, C. & Mattoso, M. (2009). Experiment Line: Software Reuse in Scientific Workflows. *SSDBM 2009, LNCS 5566*, pp. 264–272. *Springer*.
- Ogasawara, E., Castro, A., Gea, C., Borges, H., Carvalho, D., Santos, J., Bezerra, E. & Coutinho, R. (2023). Leveraging Experiment Lines to Data Analytics. *DAL/CEFET-RJ*.
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 14(2), 1137–1145.
- Guyon, I. & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. 2nd ed. *Springer*.